



Universitatea Tehnică din Cluj-Napoca

Facultatea de Automatică și Calculatoare

Structura Sistemelor de Calcul

Unitate Aritmetico-Logică (UAL) pe placa Zybo Z7

Realizat de:

Gyorgy Alina-Maria

Grupa: 30236

Îndrumător proiect:

Mureșan Mircea Paul

21 ianuarie 2026

Cuprins

1	Rezumat	3
2	Introducere	4
2.1	Obiective	5
2.2	Structura raportului	5
3	Fundamentare teoretică	6
3.1	Platforma Zynq-7020 și fluxul de co-proiectare	6
3.2	Interfața AXI4-Lite și accesul pe registre	6
3.3	UART (HTERM) și rolul aplicației software	6
3.4	Reprezentarea numerelor: întregi cu semn și floating-point	7
3.5	Operații aritmetice în ALU	7
3.6	Încadrarea proiectului și direcții viitoare	7
4	Proiectare și implementare	8
4.1	Metoda experimentală utilizată	8
4.2	Alternative de proiectare analizate	8
4.3	Arhitectura generală a sistemului	8
4.4	Componenta hardware (PL) – module și funcționalitate	9
4.4.1	Modul de top: <code>ALU.vhd</code>	9
4.4.2	Adunare/scădere pe întregi: <code>csa.vhd</code>	10
4.4.3	Înmulțire pe întregi: <code>booth_multiplier.vhd</code>	10
4.4.4	Împărțire pe întregi: <code>divider.vhd</code>	11
4.4.5	Operații pe reale: <code>fp_add.vhd</code> , <code>fp_multiplier.vhd</code>	11
4.5	Componenta software (PS) – interfață UART și control PL	11
4.6	Manual de utilizare	12
4.6.1	Cerințe	12
4.6.2	Pași de rulare	12
4.7	Limitări	12
5	Rezultate experimentale	13
5.1	Mediu de lucru și instrumente	13
5.2	Procedura de testare	13

5.3	Teste reprezentative și interpretarea rezultatelor	13
5.4	Capturi de ecran din HTERM (locuri pentru poze)	14
6	Concluzii	15

Capitolul 1

Rezumat

În cadrul acestui proiect s-a realizat o Unitate Aritmetico-Logică (UAL) implementată pe placa Zybo Z7 (Zynq-7020), care permite efectuarea operațiilor aritmetice asupra a două numere introduse de utilizator printr-o interfață serială UART (HTERM). Problema abordată a constat în proiectarea și integrarea unei soluții complete PS+PL: sistemul de procesare (ARM Cortex-A9) preia operanzii (pozitivi/negativi, întregi sau reali) și codul operației (+, -, *, /) din UART, transmite datele către ALU hardware descrisă în VHDL în partea de logică programabilă și citește rezultatul pentru a-l afișa înapoi în HTERM.

Implementarea a utilizat Vivado pentru configurarea platformei Zynq și conectarea ALU prin interfață AXI (operanzi, operație și rezultat), precum și Vitis pentru aplicația software de dialog și control, care a gestionat citirea datelor, validarea de bază și accesul la registrele AXI. Soluția a fost testată cu combinații de valori întregi pozitive/negative și valori reale (floating-point) pentru operațiile implementate. Pentru operația de împărțire pe întregi, sincronizarea a fost realizată prin semnale de status (busy/done), astfel încât rezultatul să fie citit și afișat doar după finalizarea calculului. În concluzie, proiectul demonstrează o integrare end-to-end PC–UART–PS–AXI–PL, obținând o separare clară între partea de calcul în PL și partea de control și interfață în PS.

Capitolul 2

Introducere

În zona sistemelor embedded, această tendință este vizibilă în platformele de tip SoC (System-on-Chip), unde un procesor (capabil de rularea unui software complex) este integrat împreună cu logică programabilă ce permite implementarea de circuite custom pentru sarcini critice de timp sau pentru creșterea performanței. Familia Zynq-7000 îmbină un Processing System (PS) bazat pe ARM Cortex-A9 cu Programmable Logic (PL) de tip FPGA, oferind un cadru natural pentru co-proiectarea hardware–software [3, 4, 1].

Domeniul în care se încadrează proiectul este cel al proiectării sistemelor digitale și al integrării perifericelor prin magistrale standard. O Unitate Aritmetico-Logică (UAL/ALU) reprezintă un bloc fundamental, responsabil de efectuarea operațiilor aritmetice asupra operanzilor. Într-o arhitectură Zynq, ALU poate fi implementată ca accelerator în PL și controlată din PS printr-o interfață de tip register-map. Pentru comunicația dintre PS și IP-ul din PL, un standard utilizat pe scară largă este AMBA AXI, în special varianta AXI4-Lite, potrivită pentru accesarea registrelor de control și date ale unui periferic [6, 7].

Problema abordată în acest proiect a fost realizarea unui flux complet, interactiv, de tip „calculator hardware”, în care utilizatorul introduce de la PC două numere (pozitive sau negative, întregi sau reale) și alege operația dorită (+, -, *, /) printr-o interfață serială UART, iar sistemul execută operația în ALU implementată în PL și întoarce rezultatul către utilizator. Cerința a inclus integrarea într-un sistem Zynq funcțional: recepția datelor prin UART în PS, transmiterea operandului A, a operandului B și a codului operației către PL, declanșarea execuției și citirea rezultatului. În implementarea finală, împărțirea pe reale (floating-point) nu a fost realizată; suportul floating-point a fost implementat pentru adunare/scădere/înmulțire.

Soluția propusă a urmat principiul separării rolurilor: partea de interfață și control a fost implementată în PS (aplicație în Vitis), iar partea de calcul aritmetic a fost implementată în PL (ALU în VHDL). ALU a fost împachetată ca IP custom, conectată la PS printr-o interfață AXI4-Lite, folosind un set de registre mapate în memorie: registre pentru operanzi, registru pentru operație, registru de status (busy/done pentru împărțire)

și registru pentru rezultat [3, 6, 2]. La nivel software, aplicația a gestionat dialogul cu utilizatorul în terminal (HTERM), a interpretat datele primite (inclusiv semn și format), a scris valorile în registrele hardware și a afișat rezultatul la final, utilizând driverul UART al platformei [8].

2.1 Obiective

Obiectivele principale ale proiectului au fost:

- realizarea unei interfețe UART PC–Zybo pentru introducerea operandului A, operandului B și a operației, cu afișarea rezultatului în terminal;
- proiectarea și implementarea în VHDL a unei ALU capabile să execute operațiile $+$, $-$, $*$, $/$ pentru întregi cu semn;
- extinderea suportului către valori reale utilizând floating-point (IEEE-754) pentru adunare/scădere/înmulțire;
- integrarea ALU ca IP custom cu interfață AXI4-Lite, definind registre pentru date, operație și status;
- dezvoltarea aplicației software în Vitis care scrie/citește registrele AXI, sincronizează execuția (pentru împărțire) și gestionează mesajele către utilizator;
- testarea și validarea funcționării pentru combinații reprezentative de valori și pentru cazuri limită (ex. divizare la zero la întregi).

2.2 Structura raportului

Raportul este organizat astfel: în capitolul *Fundamentare teoretică* sunt prezentate conceptele relevante pentru platformele Zynq (PS/PL), comunicarea prin AXI4-Lite, rolul UART și noțiuni despre reprezentarea numerelor și operațiile aritmetice în hardware. Capitolul *Proiectare și implementare* descrie arhitectura completă a sistemului, modulele ALU, mapa de registre și logica aplicației din Vitis. În *Rezultate experimentale* sunt prezentate scenarii de test, rezultate și observații privind depanarea. În final, capitolul *Concluzii* sintetizează realizările proiectului și propune direcții de dezvoltare viitoare, iar anexele includ materiale suport (capturi și fișiere relevante).

Capitolul 3

Fundamentare teoretică

Acest proiect se încadrează în zona de co-proiectare hardware–software pe platforme SoC cu FPGA integrat, unde controlul și interfața cu utilizatorul se realizează în software, iar calculele pot fi accelerate în hardware [3, 1]. UAL/ALU implementată în PL poate fi tratată ca accelerator conectat la PS prin registre mapate în memorie, abordare frecvent utilizată pentru prototipare și validare arhitecturală [7, 6].

3.1 Platforma Zynq-7020 și fluxul de co-proiectare

Zynq-7020 separă sistemul în PS (procesoare ARM Cortex-A9 și periferice precum UART) și PL (logică programabilă unde se implementează circuite custom în VHDL). Comunicarea PS–PL se face uzual prin AXI, astfel încât software-ul poate scrie operanzi și comenzi către hardware și poate citi rezultatele [3]. Fluxul de dezvoltare tipic este: proiectarea hardware-ului în Vivado (Block Design, IP, bitstream), exportul platformei și dezvoltarea aplicației în Vitis (citire UART, acces la registre AXI) [5, 2].

3.2 Interfața AXI4-Lite și accesul pe registre

AXI4-Lite este potrivit pentru periferice controlate prin registre, cu tranzații simple de citire/scriere. Un protocol uzual este: software-ul scrie A, B și OP, apoi citește rezultatul; pentru operații cu latență variabilă (ex. împărțire) se folosește un status (BUSY/DONE) [6, 7, 2].

3.3 UART (HTERM) și rolul aplicației software

UART oferă o metodă simplă de testare și interacțiune: utilizatorul introduce operanzii și operația în terminal, iar aplicația răspunde cu rezultatul. Pe Zynq, UART-ul PS este

accesat prin driverul XUartPs [8]. Software-ul are rol și de validare (format input, divizare la zero etc.) și coordonează accesul la registrele hardware.

3.4 Reprezentarea numerelor: întregi cu semn și floating-point

Pentru **întregi cu semn** se utilizează uzual *two's complement*, deoarece permite implementarea eficientă a adunării/scăderii. Pentru **valori reale**, proiectul folosește floating-point pe 32 biți (IEEE-754) [9]. În FPGA, suportul floating-point crește complexitatea, dar permite calcule pe reale într-o formă standardizată; în practică, acest suport poate fi implementat prin module dedicate/IP [10].

3.5 Operații aritmetice în ALU

Adunarea/scăderea se pot implementa cu arhitecturi diferite (de la ripple-carry la structuri optimizate), iar alegerea depinde de compromisurile viteză–resurse [1]. Pentru înmulțirea numerelor sembrate, algoritmul Booth reduce produsele parțiale și tratează semnul în mod natural, fiind o alegere frecventă în proiecte didactice [1]. Împărțirea este, de regulă, operația cu latență mai mare și se implementează iterativ, necesitând semnale de status pentru sincronizare [1].

3.6 Încadrarea proiectului și direcții viitoare

Proiecte educaționale similare folosesc combinația UART + AXI4-Lite + IP custom pentru a demonstra integrarea PC–PS–PL și controlul unui accelerator hardware [2, 3]. Ca direcții de extindere, se pot considera suport floating-point complet (inclusiv împărțire) și un protocol unitar de status pentru toate operațiile.

Capitolul 4

Proiectare și implementare

4.1 Metoda experimentală utilizată

Proiectul a fost realizat ca o implementare hardware–software pe Zybo Z7 (Zynq-7020). Componenta hardware (PL) a fost descrisă în VHDL și sintetizată în Vivado, iar componenta software (PS) a fost implementată în Vitis și a asigurat interfața UART (HTERM) și controlul acceleratorului din PL prin registre AXI. Validarea s-a făcut experimental, prin trimiterea de operanți și operații din terminal către placă și compararea rezultatelor cu valorile așteptate.

4.2 Alternative de proiectare analizate

- **Reprezentarea numerelor reale:** fixed-point vs floating-point. S-a ales floating-point pentru compatibilitate cu formatul IEEE-754 și pentru simplitatea utilizării la nivel de interfață (introducere valori reale) [9, 10].
- **Adder pentru întregi:** ripple-carry adder (RCA) vs implementare optimizată. S-a ales CSA pentru adunare/scădere, cu scăderea realizată prin complement față de doi ($-B + \text{carry-in}$) [1].
- **Înmulțire pe întregi:** implementare naivă vs Booth. S-a ales Booth pentru suport semnat și reducerea produselor parțiale [1].
- **Împărțire pe reale:** considerată ca extindere, însă neimplementată în această versiune. Împărțirea a fost implementată doar pentru întregi, cu un modul iterativ care semnalizează *busy/ready* [1].

4.3 Arhitectura generală a sistemului

Sistemul final este alcătuit din:

- **PC + HTERM (UART):** introducerea operandului A, operandului B și alegerea operației; afișarea rezultatului.
- **PS (Vitis):** parsare și validare input, conversie (pentru real) în format IEEE-754 pe 32 biți, scrierea registrelor către PL și citirea rezultatului.
- **PL (VHDL ALU):** execuția operațiilor (întregi și reale), generarea rezultatului și a semnalelor de status (pentru împărțire).
- **Interfață AXI4-Lite:** maparea registrelor ALU în spațiul de adrese al PS [6, 7].

4.4 Componenta hardware (PL) – module și funcționalitate

Componenta PL a fost organizată modular. Modulul de top este `ALU.vhd`, care instanțiază submodulele `csa.vhd`, `booth_multiplier.vhd`, `divider.vhd`, `fp_add.vhd` și `fp_multiplier.vhd`.

4.4.1 Modul de top: ALU.vhd

Modulul primește operanzii A, B, codul operației `opcode` și produce `RESULT` (2N biți) împreună cu un vector `status_out` (2 biți) pentru operația de împărțire (întregi).

Coduri de operație utilizate

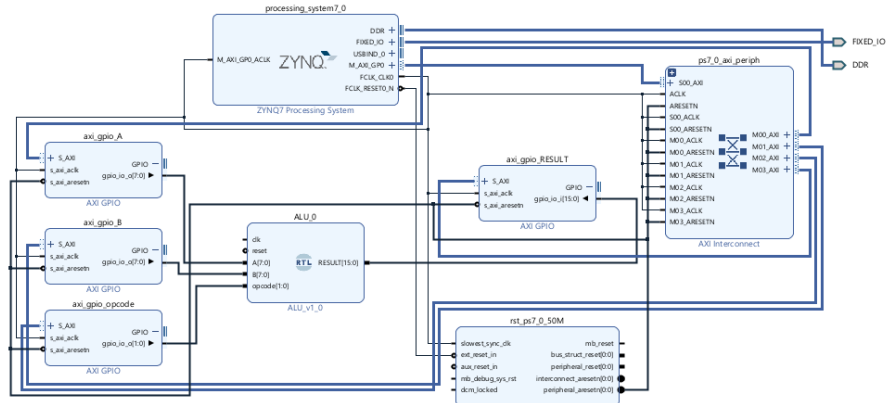
Maparea folosită este:

opcode	Operație
000	întreg: adunare ($A + B$)
001	întreg: scădere ($A - B$)
010	întreg: înmulțire ($A * B$)
011	întreg: împărțire (A / B)
100	real: adunare (FP add)
101	real: scădere (FP sub, realizată ca FP add cu semnul lui B inversat)
110	real: înmulțire (FP mul)

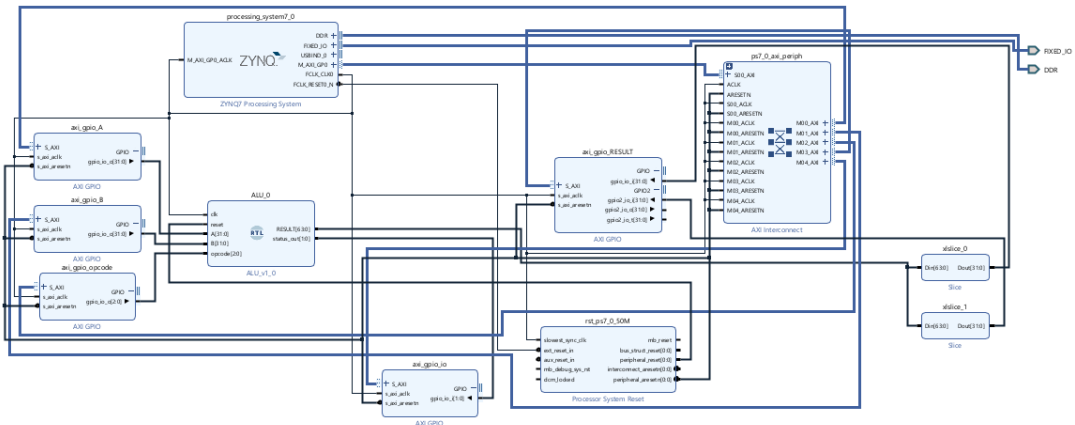
Notă: împărțirea floating-point nu a fost implementată.

Semnale de status pentru împărțirea pe întregi

- `status_out(0)`: *busy* (1 când `start_div=1` sau `div_busy=1`).
- `status_out(1)`: *done* latched (devine 1 la `div_ready=1` și rămâne 1 până la reset sau până la o nouă împărțire).



(a) Înainte de suportul floating-point (doar întregi).



(b) După adăugarea suportului floating-point (+/-/*).

Figura 4.1: Evoluția arhitecturii ALU: extinderea cu suport floating-point (fără împărțire floating-point).

4.4.2 Adunare/scădere pe întregi: csa.vhd

Operațiile de adunare și scădere sunt realizate utilizând un modul de tip CSA. Scăderea se obține prin inversarea lui B și adăugarea unui carry-in (complement față de doi), compatibil cu reprezentarea two's complement [1].

4.4.3 Înmulțire pe întregi: booth_multiplier.vhd

Înmulțirea semnată este implementată cu algoritmul Booth, care reduce numărul de adunări ale produselor parțiale prin recodificarea multiplicatorului [1].

4.4.4 Împărțire pe întregi: `divider.vhd`

Împărțirea este implementată într-un modul secvențial, controlat prin `start`. În `ALU.vhd`, operanzii sunt convertiți la valori absolute (`A_abs`, `B_abs`), semnul câtului se determină separat (`Q_sign`), iar la final rezultatul este re-semnat. Modulul expune `busy` și `ready`, utilizate pentru sincronizare.

4.4.5 Operații pe reale: `fp_add.vhd`, `fp_multiplier.vhd`

Pentru reale s-a utilizat reprezentarea floating-point pe 32 biți (IEEE-754) [9]. Operațiile implementate sunt:

- FP add prin `fp_add`;
- FP sub realizată prin inversarea bitului de semn al lui B și reutilizarea `fp_add`;
- FP mul prin `fp_multiplier` [10].

Împărțirea floating-point nu a fost implementată.

4.5 Componenta software (PS) – interfață UART și control PL

Aplicația din PS gestionează dialogul UART și controlul acceleratorului din PL:

1. citirea operandului A;
2. citirea operandului B;
3. citirea operației și stabilirea `opcode`;
4. conversie: întregi ca semnat pe N biți; reale ca IEEE-754 pe 32 biți;
5. scrierea registrelor ALU (A, B, `opcode`);
6. pentru împărțire pe întregi: polling pe status până la *done*; apoi citirea rezultatului;
7. afișarea rezultatului în HTERM.

Accesul serial s-a realizat prin driverul XUartPs [8], iar accesul la perifericul din PL prin scrieri/citiri memory-mapped (AXI4-Lite) [6, 7].

4.6 Manual de utilizare

4.6.1 Cerințe

- PC cu terminal serial (HTERM sau echivalent);
- Vivado (generare bitstream) și Vitis (compilare aplicație) pentru programarea plăcii;
- conexiune UART prin USB/UART a plăcii Zybo Z7 [5].

4.6.2 Pași de rulare

1. Se programează placa cu bitstream-ul generat în Vivado.
2. Se rulează aplicația din Vitis pe PS.
3. În HTERM se selectează portul serial și parametrii UART folosiți în proiect.
4. Se introduc operandul A și operandul B, apoi operația.
5. Se afișează rezultatul; pentru împărțire pe întregi aplicația așteaptă finalizarea (done).

4.7 Limitări

În această versiune, împărțirea floating-point nu este implementată. De asemenea, semnalizarea *busy/done* este utilizată explicit doar pentru împărțirea pe întregi.

Capitolul 5

Rezultate experimentale

5.1 Mediu de lucru și instrumente

Implementarea a fost realizată în VHDL (componenta PL) și C (componenta PS). Pentru proiectare și integrare s-au utilizat Vivado (Block Design, generare bitstream) și Vitis (aplicația UART și controlul registrelor AXI). Testarea interactivă s-a realizat prin terminal serial HTERM. Platforma hardware utilizată a fost Zybo Z7 (Zynq-7020).

- Sistem de operare: Windows 11
- Vivado: v2024.1
- Vitis: v2024.1
- Terminal serial: HTERM

5.2 Procedura de testare

Testarea a urmărit validarea fluxului complet PC–UART–PS–AXI–PL și verificarea corectitudinii operațiilor pentru:

- întregi cu semn: adunare, scădere, înmulțire, împărțire;
- reale (floating-point): adunare, scădere, înmulțire (fără împărțire floating-point).

5.3 Teste reprezentative și interpretarea rezultatelor

În testele de mai jos, codul operației a fost interpretat astfel: **op=0** adunare, **op=1** scădere, **op=2** înmulțire, **op=3** împărțire (întregi). Valorile din coloana *rezultat afișat (raw)* reprezintă ce a fost afișat în momentul testării.

Tabela 5.1: Teste reprezentative pe variante și observații (preluate din notepadul de testare).

Variantă	A	B	op	Rezultat raw	Așteptat	Observație
v2	2	3	0	5	5	adunare corectă
v2	10	2	2	20	20	înmulțire corectă
v2	-2	1	0	255	-1	interpretare unsigned /lățime nepotrivită (negativ → număr mare)
v2	4	-2	0	65282	2	rezultat negativ afișat ca număr mare (unsigned)
v2	-4	-4	2	16	16	înmulțire corectă pe semn (Booth)
v2	6	3	3	128	2	împărțirea nu era încă interpretată/transferată corect în varianta respectivă
v4	2	-1	0	1	1	după corecții parțiale: adunare corectă
v4	2	-1	2	65534	-2	negativ afișat ca 0xFFFFE (citire ca unsigned)

Notă de interpretare (semn vs unsigned). În etapele intermediare, erorile au apărut când operandul sau rezultatul era tratat ca **unsigned** în PS (Vitis) în loc de **signed**. Astfel, valori negative apăreau ca numere mari pozitive. Corecția a constat în utilizarea tipurilor semnate și a conversiilor corecte la scrierea/citirea registrelor și la afișare.

5.4 Capturi de ecran din HTERM (locuri pentru poze)

```

1   5   10  15  20  25  30  35  40  45  50  55  60  65  70  75  80
--- Selectati tipul de date ---
1 - Int
2 - Float
> Ai ales: 1v
Operand A (int): Ai introdus A = -3v
Operand B (int): Ai introdus B = 4v
OpCode (0-adunare, 1-scadere, 2-inmultire, 3-impartire): Ai introdus OpCode = 2v
RESULTAT INT 32: -12v

--- Selectati tipul de date ---
1 - Int
2 - Float
> Ai ales: 1v
Operand A (int): Ai introdus A = -4v
Operand B (int): Ai introdus B = 2v
OpCode (0-adunare, 1-scadere, 2-inmultire, 3-impartire): Ai introdus OpCode = 3v
RESULTAT INT 32: -2v

```

```

1   5   10  15  20  25  30  35  40  45  50  55  60  65  70  75
--- Selectati tipul de date ---
1 - Int
2 - Float
> Ai ales: 2v
Operand A (float): Ai introdus A = 1.500000v
Operand B (float): Ai introdus B = -3.000000v
OpCode (4-adunare, 5-scadere, 6-inmultire): Ai introdus OpCode = 6v
RESULTAT FLOAT: -4.500000 (Hex: 0xC0900000)v

--- Selectati tipul de date ---
1 - Int
2 - Float
> Ai ales: 2v
Operand A (float): Ai introdus A = 5.750000v
Operand B (float): Ai introdus B = -3.540000v
OpCode (4-adunare, 5-scadere, 6-inmultire): Ai introdus OpCode = 4v
RESULTAT FLOAT: 2.210000 (Hex: 0x40D70A4)v

```

(a) Test pe întregi (semn corect). (b) Test floating-point (FP add/sub/mul).

Figura 5.1: Exemple de rulare în HTERM: întregi vs. floating-point.

Capitolul 6

Concluzii

În acest proiect am realizat o Unitate Aritmetico-Logică pe placa Zybo Z7 (Zynq-7020), controlată din PS prin AXI și utilizată interactiv prin UART (HTERM). Sistemul a primit de la utilizator doi operanzi și o operație, a transmis datele către ALU hardware din PL și a afișat rezultatul înapoi în terminal. Obiectivele principale au fost atinse: integrarea completă PS–PL, implementarea operațiilor pe întregi cu semn (adunare, scădere, înmulțire, împărțire) și extinderea pentru numere reale folosind floating-point pentru adunare, scădere și înmulțire.

Contribuțiile proprii cele mai importante au fost proiectarea modulară a componentelor hardware (CSA pentru adunare/scădere, înmulțire semnată cu Booth și un modul secvențial pentru împărțire) și integrarea lor într-un modul ALU unic cu selecție pe opcode. De asemenea, am realizat partea software în Vitis pentru citirea datelor prin UART, conversiile necesare și controlul execuției prin registre mapate AXI. Un aspect important în procesul de dezvoltare a fost depanarea conversiilor numerelor cu semn: folosirea tipului `unsigned` în loc de `signed` a dus la rezultate eronate pentru valori negative, problemă identificată prin teste simple și corectată prin conversii semnate corecte.

Avantajele soluției sunt separarea clară între control (PS) și calcul (PL), execuția rapidă a operațiilor în hardware și posibilitatea de extindere prin adăugarea de module. Dezavantajele sunt complexitatea mai mare față de o implementare exclusiv software și faptul că, în versiunea actuală, împărțirea floating-point nu este implementată. În plus, sincronizarea prin semnale de status este utilizată explicit doar pentru împărțirea pe întregi.

Ca dezvoltări viitoare, se pot implementa: împărțirea floating-point, un protocol unitar de *busy/done* pentru toate operațiile și tratări mai complete ale erorilor (overflow, NaN/Inf în floating-point), respectiv optimizări de performanță prin pipeline sau IP-uri dedicate.

Bibliografie

- [1] Universitatea Tehnică din Cluj-Napoca (UTCN), Facultatea de Automatică și Calculatoare, *Structura Sistemelor de Calcul – suport de curs*, material didactic intern.
- [2] Universitatea Tehnică din Cluj-Napoca (UTCN), Facultatea de Automatică și Calculatoare, *Structura Sistemelor de Calcul – laborator/proiect (Zynq, AXI, Vivado/-Vitis)*, material didactic intern.
- [3] AMD (Xilinx), *Zynq-7000 SoC Technical Reference Manual (UG585)*, documentație oficială. <https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM>
- [4] AMD (Xilinx), *Zynq-7000 SoC Data Sheet: Overview (DS190)*, documentație oficială. <https://docs.amd.com/v/u/en-US/ds190-Zynq-7000-Overview>
- [5] Digilent, *Zybo Z7 Reference Manual*, documentație oficială. <https://digilent.com/reference/programmable-logic/zybo-z7/reference-manual>
- [6] Arm, *AMBA AXI and ACE Protocol Specification (AXI4/AXI4-Lite)*, documentație oficială. <https://developer.arm.com/documentation/ih0022/latest>
- [7] Xilinx, *AXI Reference Guide (UG761)*, documentație oficială. https://docs.xilinx.com/r/en-US/ug761_axi_reference_guide
- [8] AMD (Xilinx), *Vitis Drivers API: XUartPs (UARTPS)*, documentație oficială. https://xilinx.github.io/embeddedsw.github.io/uartps/doc/html/api/xuartps_8h.html
- [9] IEEE, *IEEE Standard for Floating-Point Arithmetic (IEEE 754)*, pagină oficială standard. <https://standards.ieee.org/ieee/754/6210/>
- [10] AMD (Xilinx), *Floating-Point Operator (LogiCORE IP) Product Guide (PG060)*, documentație oficială. <https://docs.xilinx.com/r/en-US/pg060-floating-point>